

1	Create a Kanban Board to Visualize the Tasks. • Create Columns for To Do, In-Progress and Done. • Add at least 5 Sample Tasks • Move the Tasks across the Columns to Simulate the Workflow.	Jira or Trello	CO1
---	---	----------------	-----

Using Jira

Objective

To create a Kanban Board with *To Do*, *In-Progress*, and *Done* columns and simulate task movement.

Step 1: Login

1. Go to jira.atlassian.com
2. Login to Jira account.

Step 2: Create Project

1. Click Create Project
2. Select Kanban project
3. Name: Kanban Workflow Project
4. Click Create.

Step 3: Verify Columns

Ensure board has:

- To Do
- In Progress
- Done

(If not, configure columns in board settings.)

Step 4: Create Issues (Tasks)

Click Create Issue and add:

1. Collect project requirements
2. Design UI mockups
3. Develop login module
4. Test application
5. Prepare final report

Step 5: Move Issues

Drag issues:

- First two → In Progress
- Then move one → Done

Step 6: Final Simulation

- Design UI mockups → Done
- Develop login module → In Progress

Using Trello

Step 1: Login

1. Open browser → go to trello.com
2. Login using Google / email account.

Step 2: Create a Board

1. Click **Create** → **Create Board**
2. Name it: **Kanban Workflow Board**
3. Click **Create**.

Step 3: Create Columns (Lists)

Rename default lists as:

- To Do
- In-Progress
- Done

(Delete extra lists if any.)

Step 4: Add Sample Tasks

Under **To Do**, add these 5 cards:

1. Collect project requirements
2. Design UI mockups
3. Develop login module
4. Test application
5. Prepare final report

Step 5: Move Tasks to In-Progress

Drag and drop:

- Collect project requirements
- Design UI mockups

➡ from **To Do** to **In-Progress**

Step 6: Move Completed Tasks to Done

After completion:

Drag **Collect project requirements** → to **Done**

Step 7: Continue Workflow Simulation

- Move **Design UI mockups** → Done
- Move **Develop login module** → In-Progress

Final Board Example

To Do	In-Progress	Done
Test application	Develop login module	Collect project requirements
Prepare final report		Design UI mockups

Result

Thus, a Kanban board was successfully created using Jira/Trello and tasks were moved across columns to simulate workflow.

2	Sketch a Simple Prototype of a Bus Ticket Booking System using Figma Tool	Figma	CO1
---	---	-------	-----

Objective

To design a simple interactive prototype of a Bus Ticket Booking System using Figma.

Step 1: Login to Figma

1. Open browser → go to **figma.com**
2. Click **Login / Sign Up**
3. Login using Google or email.

Step 2: Create a New Design File

1. Click **New Design File**
2. Rename file as:
Bus Ticket Booking Prototype

Step 3: Create Frame (Mobile Screen)

1. Click **Frame Tool (F)**
2. Select device: **Android / iPhone (375 × 812)**
3. This represents the mobile app screen.

Step 4: Design Home Screen

Add these UI elements:

Header

- Text: **Bus Booking App**

Input Fields

- From City
- To City
- Travel Date

Button

- **Search Buses**

Arrange neatly using rectangles and text.

Step 5: Design Bus List Screen

1. Duplicate first frame (Ctrl + D)
2. Add:
 - Bus Name
 - Time
 - Price
 - **Book Now** buttonCreate 2–3 sample bus listings.

Step 6: Design Seat Selection Screen

Duplicate frame again:

Add:

- Seat grid (rectangles as seats)
- Selected seat in different color
- **Proceed to Payment** button

Step 7: Design Payment Screen

Duplicate frame:

Add:

- Ticket summary
- Total price
- Payment method dropdown
- **Pay Now** button

Step 8: Create Confirmation Screen

Duplicate frame:

Add:

- ✓ Booking Confirmed
- Ticket number
- Travel details
- **Download Ticket** button

Step 9: Create Prototype Links

1. Click **Prototype tab**
2. Connect:
 - Search Buses → Bus List Screen

- Book Now → Seat Selection
- Proceed → Payment
- Pay Now → Confirmation

Step 10: Preview Prototype

1. Click **Play ► icon**
2. Test full booking flow.

Step 11: Save and Share

1. Click **Share**
2. Set permission to *Anyone with link – View*
3. Copy link.

Result

Thus, a simple prototype of a Bus Ticket Booking System was successfully designed using Figma.

3	The stakeholders have conflicting views on the user interface design for an E-Commerce mobile app. Create a prototype using Figma tool to discuss with the stakeholders to get their feedback and approval.	Figma	CO1
---	---	-------	-----

Objective

To design a clickable UI prototype of an E-Commerce mobile application using Figma in order to present multiple interface options and collect stakeholder feedback for approval.

Step 1: Login to Figma

1. Open **figma.com**
2. Login using Google or Email.

Step 2: Create New Design File

1. Click **New Design File**
2. Rename file as:
E-Commerce UI Prototype – Stakeholder Review

Step 3: Create Mobile Frame

1. Select **Frame Tool (F)**
2. Choose device: **iPhone 14 / Android (375×812)**
3. This is the app screen.

Step 4: Create Home Screen – Version A

Design a **clean minimal UI**:

Elements:

- App logo + title
- Search bar
- Banner image
- Category icons
- Product cards
- Bottom navigation bar

Label frame as:

🔗 **Home Screen – Version A (Minimal UI)**

Step 5: Duplicate Frame for Version B

Duplicate frame (Ctrl + D)

Modify design to:

- Dark theme
- Larger banners
- Sidebar menu icon
- Bigger product cards

Label as:

🔗 **Home Screen – Version B (Visual UI)**

This helps compare conflicting stakeholder preferences.

Step 6: Create Product Detail Screen

Duplicate one frame:

Add:

- Product image
- Price
- Rating
- Description
- **Add to Cart** button
- **Buy Now** button

Step 7: Create Cart Screen

Duplicate frame:

Add:

- Product list
- Quantity buttons
- Total price
- **Checkout** button

Step 8: Create Payment Screen

Add:

- Address section
- Payment options
- **Confirm Order** button

Step 9: Create Order Confirmation Screen

Add:

- ✓ Order Successful
- Order ID

- Delivery date
- **Continue Shopping** button

Step 10: Create Prototype Navigation

Open **Prototype Tab** and connect:

From	To
Version A Home → Product	Product Detail
Version B Home → Product	Product Detail
Add to Cart	Cart Screen
Checkout	Payment Screen
Confirm Order	Confirmation Screen

Step 11: Preview Prototype

1. Click **Play** ►
2. Test both Version A and Version B flows.

Step 12: Share for Stakeholder Feedback

1. Click **Share**
2. Set access to:
Anyone with link → Can comment
3. Copy link and share with stakeholders.

Step 13: Collect Feedback

Stakeholders can:

- Add comments on screens
- Suggest UI changes
- Approve preferred version

Result

Thus, an interactive E-Commerce mobile app prototype was created using Figma to resolve stakeholder UI conflicts and obtain design approval.

4	The string is a sequence of characters placed in double quotes (" "). Performing different operations on string data is called String Handling. Strings are immutable. Whenever a change to a String is made, an entirely new String is created. If we want to store a group of characters we can use a char array. String provides various methods to perform different operations on strings. Using Raptor draw a flowchart to display the total number of characters in the string and returns it.	Raptor	CO1
---	---	--------	-----

Objective

To design a RAPTOR flowchart that accepts a string, calculates the total number of characters in the string, and displays the result.

Algorithm

1. Start
2. Read the string
3. Find length of the string
4. Display the length
5. Stop

Step 1: Open RAPTOR

1. Open RAPTOR application
2. Click **New Flowchart**

Step 2: Insert Start Symbol

- RAPTOR automatically provides **Start**.

Step 3: Input the String

1. Drag **Input symbol**
2. Write:

Input str

Step 4: Process to Find Length

1. Drag **Assignment** symbol
2. Write:
 $\text{count} \leftarrow \text{length}(\text{str})$

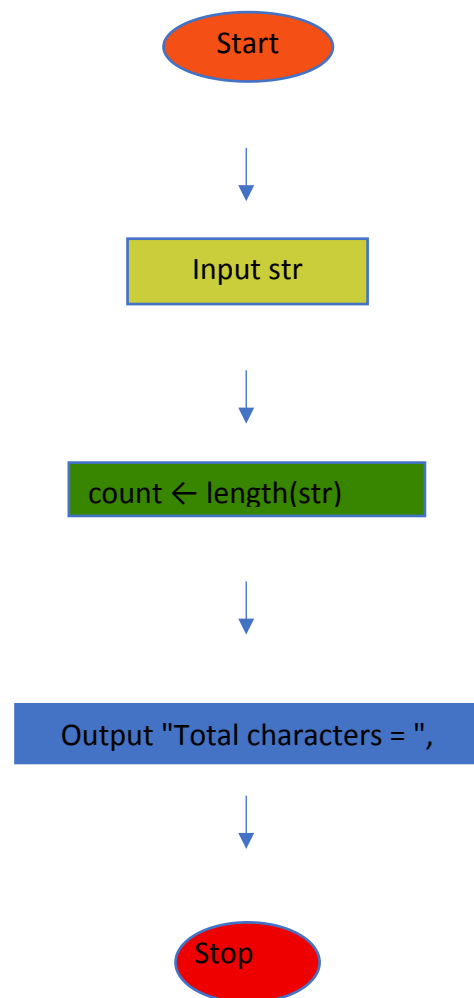
Step 5: Display Result

1. Drag **Output** symbol
2. Write:
Output "Total characters = ", count

Step 6: Stop

- RAPTOR automatically provides **Stop**.

Final Flowchart Structure



Step 7: Run the Program

1. Click **Run** ►
2. Enter string:
Example: "Computer"
3. Output will be:
Total characters = 8

Result

Thus, a RAPTOR flowchart was successfully designed to calculate and display the number of characters in a string.

5	Using Raptor – Draw and validate a flowchart for a given string of lower case English alphabets. One can choose any two characters in the string and replace all the occurrences of the first character with the second character and replace all the occurrences of the second character with the first character. Using Raptor draw and validate the flowchart lexicographically smallest string that can be obtained by doing this operation at most once.	Raptor	CO1
---	---	--------	-----

Objective

To design and validate a RAPTOR flowchart that transforms a given lowercase string into the lexicographically smallest possible string by swapping two characters at most once.

Algorithm

1. Read string str
2. Find the first position i where a smaller character exists later
3. Select two characters c1 and c2
4. Replace all c1 with c2 and all c2 with c1
5. Display the modified string

Step-by-Step RAPTOR Instructions

♦ Step 1: Start RAPTOR

Open RAPTOR → Click **New Flowchart**

♦ Step 2: Input String

Insert **Input symbol**

Input str

♦ Step 3: Initialize Variables

Insert **Assignment symbol**

$n \leftarrow \text{length}(\text{str})$

found $\leftarrow$ false

♦ **Step 4: Outer Loop (i)**

Insert **Loop symbol (For loop)**

for i $\leftarrow$ 1 to n

♦ **Step 5: Inner Loop (j)**

Inside loop:

for j $\leftarrow$ i+1 to n

♦ **Step 6: Decision to Find Smaller Character**

Insert **Decision symbol**

if str[j] < str[i] and found = false

♦ **Step 7: Store Swap Characters**

Inside IF:

c1 $\leftarrow$ str[i]

c2 $\leftarrow$ str[j]

found $\leftarrow$ true

♦ **Step 8: End Both Loops**

Exit loops once found = true

♦ **Step 9: Replace Characters in String**

Insert loop:

for k $\leftarrow$ 1 to n

Inside loop:

if str[k] = c1

 str[k] $\leftarrow$ c2

else if $\text{str}[k] = c_2$

$\text{str}[k] \leftarrow c_1$

end if

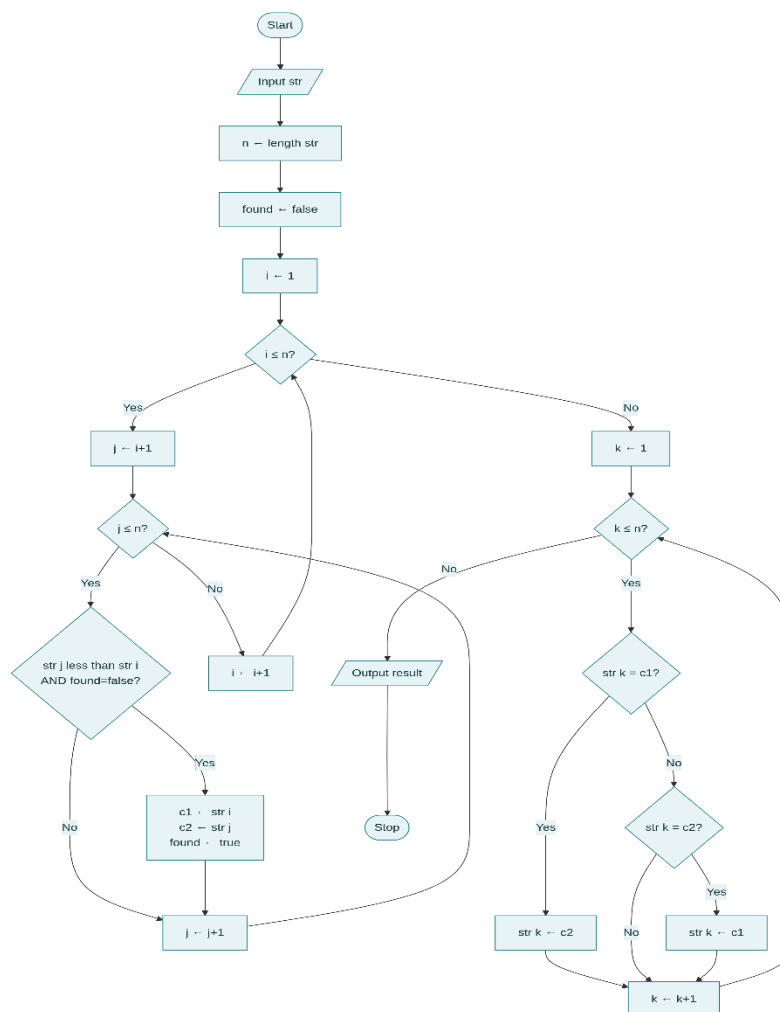
♦ Step 10: Output Result

Insert **Output symbol**

Output "Lexicographically smallest string = ", str

♦ Step 11: Stop

Flowchart



Result

Thus, a RAPTOR flowchart was successfully designed and validated to generate the lexicographically smallest string using at most one character swap.

6	An online food delivery platform enables customers to browse restaurants, view menus, place food orders, make online payments, and track delivery status. Restaurants use the system to manage their menus and update order status, while delivery agents are responsible for accepting delivery requests and updating delivery progress. Design a Class Diagram, Use case Diagram and Activity Diagram for the Online Food Delivery System showing the relevant classes, their attributes, methods, and relationships.	Umbrello	CO1
---	---	----------	-----

Objective

To design UML diagrams (Class, Use Case, and Activity) for an Online Food Delivery System using Umbrello, representing system functionality, structure, and workflow.

System Description (Short)

The Online Food Delivery System allows:

- Customers → browse restaurants, order food, make payments, track delivery
- Restaurants → manage menu and update order status
- Delivery Agents → accept orders and update delivery progress

♦ PART 1: CLASS DIAGRAM

Step 1: Open Umbrello

1. Open **Umbrello UML Modeller**
2. Click **File → New**

Step 2: Select Class Diagram

1. In left panel → Right click **Logical View**
2. Select **New Diagram → Class Diagram**
3. Rename as:
 📁 FoodDelivery_ClassDiagram

Step 3: Add Classes

Click **Class icon** and create:

1. Customer
2. Restaurant
3. Menu
4. FoodItem
5. Order
6. Payment
7. DeliveryAgent

Step 4: Add Attributes & Methods

Customer

- customerId
- name
- address
- phone
- Methods:
- browseRestaurants()
- placeOrder()
- makePayment()
- trackOrder()

Restaurant

- restaurantId
- name
- location
- Methods:
- updateMenu()

- acceptOrder()

Menu

- menuId
Methods:
- showItems()

FoodItem

- itemId
- itemName
- price

Order

- orderId
- status
Methods:
- updateStatus()

Payment

- paymentId
- amount
- paymentStatus
Methods:
- processPayment()

DeliveryAgent

- agentId
- name
Methods:
- acceptDelivery()
- updateDeliveryStatus()

Step 5: Draw Relationships

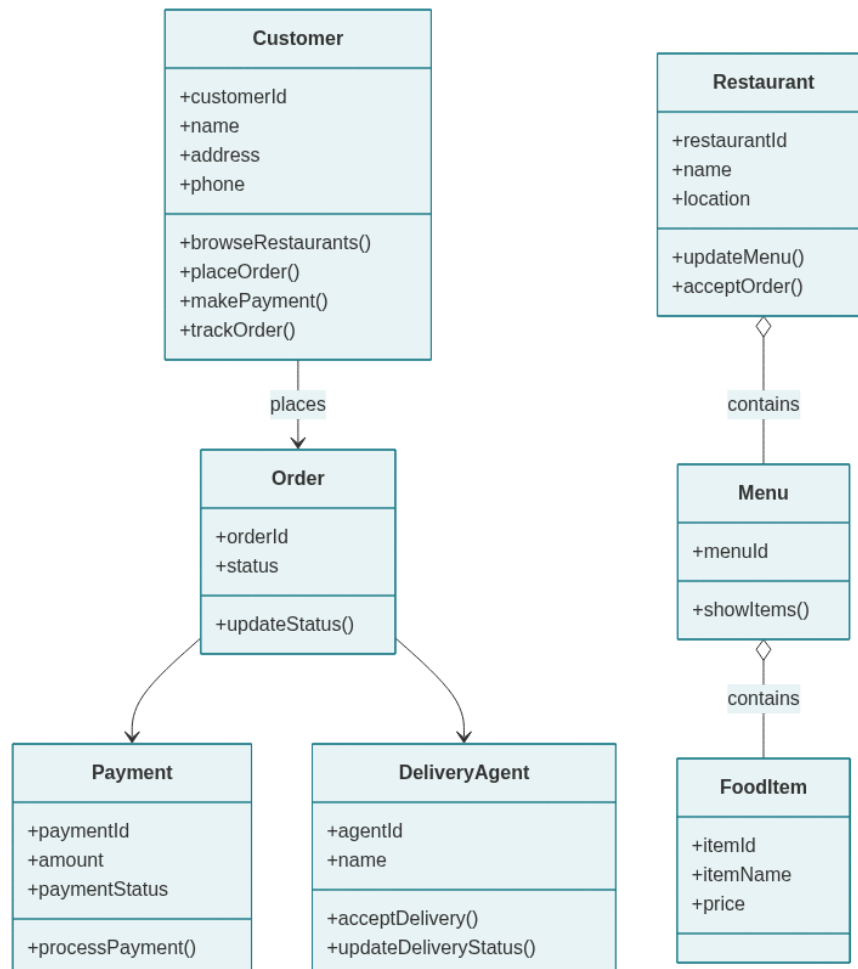
Use **Association / Aggregation** lines:

- Customer → Order
- Restaurant → Menu
- Menu → FoodItem
- Order → Payment

- Order → DeliveryAgent

Step 6: Save Diagram

File → Save as FoodDelivery_ClassDiagram.uxf



◆ PART 2: USE CASE DIAGRAM

Step 1: Create Use Case Diagram

Right click **Use Case View** → **New Diagram**

Rename as: FoodDelivery_UseCaseDiagram

Step 2: Add Actors

Add:

- Customer
- Restaurant
- Delivery Agent

Step 3: Add Use Cases

Customer Use Cases

- Browse Restaurants
- View Menu
- Place Order
- Make Payment
- Track Delivery

Restaurant Use Cases

- Manage Menu
- Update Order Status

Delivery Agent Use Cases

- Accept Delivery
- Update Delivery Status

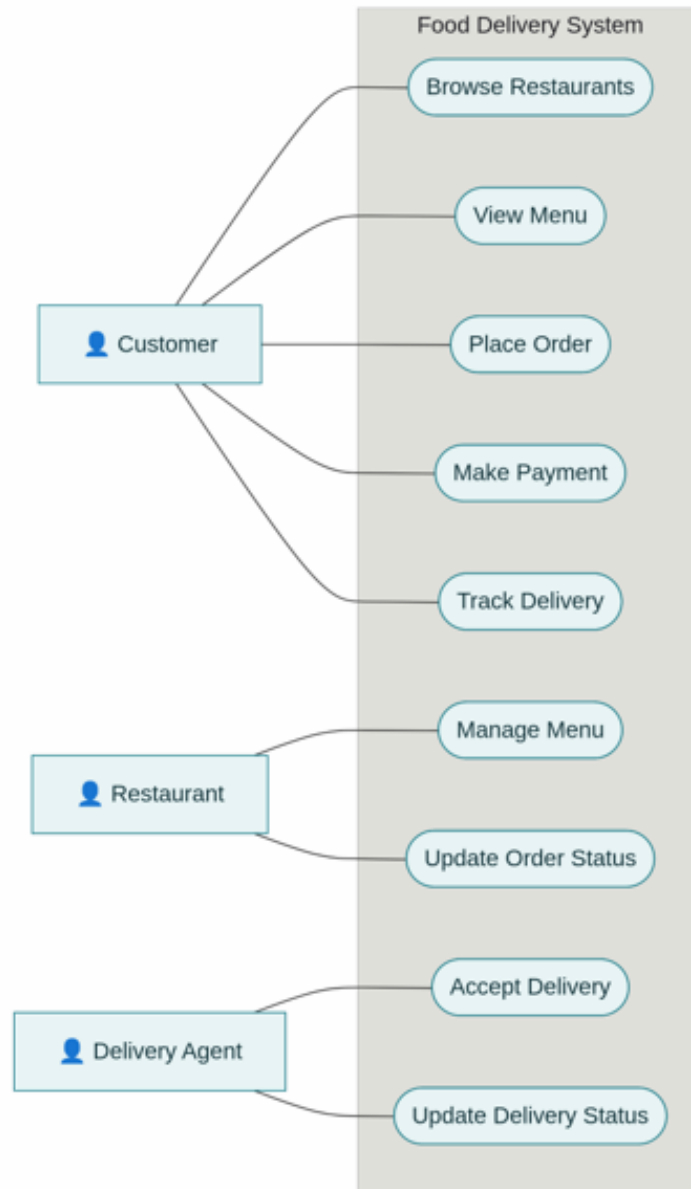
Step 4: Connect Actors to Use Cases

Use **Association line**:

- Customer → all customer use cases
- Restaurant → manage menu, update order
- Delivery Agent → delivery tasks

Step 5: Save Diagram

Save as: FoodDelivery_UseCaseDiagram.uxf



◆ PART 3: ACTIVITY DIAGRAM

Step 1: Create Activity Diagram

Right click **Logical View** → **New Diagram** → **Activity Diagram**

Rename as: FoodDelivery_ActivityDiagram

Step 2: Add Activities in Sequence

1. Start
2. Browse Restaurants
3. View Menu
4. Select Food
5. Place Order
6. Make Payment
7. Order Confirmed
8. Restaurant Prepares Food
9. Delivery Agent Picks Order
10. Deliver Food
11. Order Completed
12. Stop

Step 3: Connect Flow

Use **Control Flow arrows** to connect each activity in order.

Step 4: Save Diagram

Save as: FoodDelivery_ActivityDiagram.uxf

Result

Thus, Class Diagram, Use Case Diagram, and Activity Diagram for the Online Food Delivery System were successfully designed using Umbrello UML Modeller.

7	A college employs a large number of security personnel who are assigned duties at various locations across the campus. Each security person follows a duty schedule that includes date, duty start time, duty end time, and assigned location, ensuring that every campus location is covered. If a security person takes leave, it is recorded manually, and at the end of each month, salaries are calculated based on duties performed and leaves taken. Since manual handling of duty allocation, leave management, and salary calculation is time-consuming and complex, the college plans to develop an Online Security Management System to automate these activities. Develop Class Diagram, Use case Diagram and Activity Diagram to represent the workflow of the Online Security Management System.	Umbrello	CO1
---	--	----------	-----

Objective

To design UML diagrams for an Online Security Management System that automates duty allocation, leave management, and salary calculation of security personnel.

System Description (Short)

The system manages:

- Security personnel details
- Duty schedules
- Leave records
- Salary calculation

It automates manual processes for efficiency and accuracy.

♦ PART 1 — CLASS DIAGRAM

Step 1: Open Umbrello

Open **Umbrello UML Modeller**

Click **File → New**

Step 2: Create Class Diagram

1. Right-click **Logical View**

2. Click **New Diagram → Class Diagram**
3. Rename as:
SecurityManagement_ClassDiagram

Step 3: Create Classes

Add these classes:

1. SecurityStaff
2. DutySchedule
3. Location
4. Leave
5. Salary
6. Admin

Step 4: Add Attributes and Methods

◆ SecurityStaff

Attributes:

- staffId
- name
- phone
- designation

Methods:

- viewDuty()
- applyLeave()

◆ DutySchedule

Attributes:

- dutyId
- date
- startTime
- endTime

Methods:

- assignDuty()
- updateDuty()

◆ Location

Attributes:

- locationId

- locationName

Methods:

- updateLocation()

◆ Leave

Attributes:

- leaveId
- leaveDate
- reason

Methods:

- approveLeave()

◆ Salary

Attributes:

- salaryId
- month
- amount

Methods:

- calculateSalary()

◆ Admin

Attributes:

- adminId
- name

Methods:

- assignDuty()
- manageLeave()
- generateSalary()

Step 5: Draw Relationship

Use Association / Aggregation:

From	To	Relationship
------	----	--------------

SecurityStaff	DutySchedule	Association
---------------	--------------	-------------

DutySchedule	Location	Association
SecurityStaff	Leave	Association
SecurityStaff	Salary	Association
Admin	SecurityStaff	Association

Step 6: Save Diagram

File → Save

♦ PART 2 — USE CASE DIAGRAM

Step 1: Create Use Case Diagram

Right-click **Use Case View** → **New Diagram**

Rename as:

SecurityManagement_UseCaseDiagram

Step 2: Add Actors

Add:

- Admin
- Security Staff

Step 3: Add Use Cases

Admin Use Cases

- Assign Duty
- Update Duty
- Approve Leave
- Calculate Salary
- Generate Salary Report

Security Staff Use Cases

- View Duty Schedule
- Apply Leave
- View Salary

Step 4: Connect Actors to Use Cases

Use association lines between:

- Admin → admin use cases
- Security Staff → staff use cases

Step 5: Save Diagram

♦ PART 3 — ACTIVITY DIAGRAM

Step 1: Create Activity Diagram

Right-click **Logical View** → **New Diagram** → **Activity Diagram**

Rename as:

SecurityManagement_ActivityDiagram

Step 2: Add Activities in Order

1. Start
2. Admin Assigns Duty
3. Security Staff Views Duty
4. Security Staff Performs Duty
5. Apply Leave (if required)
6. Admin Approves Leave
7. Monthly Duty Completion
8. Calculate Salary
9. Generate Salary
10. Security Staff Views Salary
11. Stop

Step 3: Add Decision Node

After **Apply Leave**

Decision:

Leave Approved?

- Yes → Continue
- No → Duty Continues

Step 4: Connect All Activities with Arrows

Use control flow arrows to show sequence.

Step 5: Save Diagram

Result

Thus, Class Diagram, Use Case Diagram, and Activity Diagram for the Online Security Management System were successfully designed using Umbrello UML Modeller.

	Cyclomatic Complexity in Software Testing is a testing metric		
--	--	--	--

8	used for measuring the complexity of a software program. It is a quantitative measure of independent paths in the source code of a software program. Cyclomatic complexity can be calculated by using control flow graphs or with respect to functions, modules, methods or classes within a software program. Cyclomatic complexity for a flow graph G is $V(G)=E-N+2$. Cyclomatic complexity is a software metric (measurement), used to indicate the complexity of a program, Cyclomatic complexity = $E - N + P$ where, E = number of edges in the flow graph, N = number of nodes in the flow graph, P = number of nodes that have exit points. Find Cyclomatic Complexity for a graph having number of edges as 17, number of nodes as 13 and number of predicate nodes in the flow graph as 5.	Raptor	CO1
---	---	--------	-----

Given Data

Edges (E) = **17**

Nodes (N) = **13**

Predicate nodes (P) = **5**

Formula used in Software Testing

Two common formulas:

1. **$V(G) = E - N + 2$**
2. **$V(G) = P + 1$**

We will compute using both to verify.

♦ Manual Calculation

Method 1:

$$V(G) = E - N + 2$$

$$V(G) = 17 - 13 + 2$$

$$V(G) = 6$$

Method 2:

$$V(G) = P + 1$$

$$V(G) = 5 + 1$$

$$V(G) = 6$$

Cyclomatic Complexity = 6

Aim

To calculate Cyclomatic Complexity using RAPTOR.

Step 1: Open RAPTOR

Open RAPTOR Flowchart Tool.

Step 2: Insert Input Symbols

Add **three Input symbols**:

1. Input E
2. Input N
3. Input P

Set prompts:

Enter number of edges:

Enter number of nodes:

Enter number of predicate nodes:

Step 3: Insert Assignment Symbol (Formula 1)

Add an **Assignment symbol**:

$$V1 = E - N + 2$$

Step 4: Insert Assignment Symbol (Formula 2)

Add another **Assignment symbol**:

$$V2 = P + 1$$

Step 5: Insert Output Symbols

Add Output symbols:

Cyclomatic Complexity using $E - N + 2 = V1$

Cyclomatic Complexity using $P + 1 = V2$

Step 6: Flowchart Order

Start



Input E



Input N



Input P



$$V1 = E - N + 2$$



$$V2 = P + 1$$



Output V1



Output V2

↓

End

Step 7: Sample Input for Lab

$E = 17$

$N = 13$

$P = 5$

Output:

Cyclomatic Complexity using $E - N + 2 = 6$

Cyclomatic Complexity using $P + 1 = 6$

Result

The Cyclomatic Complexity of the given flow graph is:

$$V(G) = 6$$

9	Create a Scrum Project in Jira. • Add a backlog with at least 5 items (e.g., "Create user registration page", "Develop API for login"). • Prioritize the backlog and create a 1-week sprint. • Move backlog items into the sprint and start the sprint. • Finally show the Screenshot of the sprint board at the start and end of the sprint.	Jira	CO2
---	---	------	-----

Aim

To create a Scrum project in Jira, add backlog items, prioritize them, create a sprint, start the sprint, and track progress using the sprint board.

Requirements

- Jira account (Atlassian Cloud or Server)
- Browser access

Step 1: Create a Scrum Project

1. Login to **Jira**.
2. Click **Projects** → **Create Project**.
3. Choose **Scrum Software Development** template.
4. Click **Use template**.
5. Select **Team-managed project** (recommended for lab).
6. Enter:
 - Project Name: Library Scrum Project
 - Key: LSP
7. Click **Create Project**.

Scrum project is created.

Step 2: Open Backlog

1. From left menu → Click **Backlog**.
2. You will see **Product Backlog** and **Sprint area**.

Step 3: Add Backlog Items (Minimum 5)

Click **Create issue** or **+ Create** and add these as **Story** type:

Priority	Backlog Item
High	Create user registration page
High	Develop API for login
Medium	Design home page UI
Medium	Implement password reset
Low	Write unit test cases

Step 4: Prioritize the Backlog

1. In Backlog view, drag issues up/down.
2. Arrange in this order:
 - Create user registration page
 - Develop API for login
 - Design home page UI
 - Implement password reset
 - Write unit test cases

Top = Highest priority.

Step 5: Create a 1-Week Sprint

1. Click **Create Sprint** in Backlog.
2. Rename sprint as:
3. Sprint 1 – User Authentication Module
4. Set duration to **1 week** when starting sprint.

Step 6: Move Backlog Items to Sprint

Drag all 5 backlog items into **Sprint 1** section.

Now Sprint contains all selected tasks.

Step 7: Start the Sprint

1. Click **Start Sprint**.
2. Set:
 - Sprint duration: 1 week
 - Start date: Today
3. Click **Start**.

Sprint is now active.

Step 8: Sprint Board at Start (Screenshot 1)

1. Click **Active Sprint / Board**.
2. All tasks will be in **To Do** column.

Take Screenshot – Sprint Board at Start

(For lab submission: paste this screenshot)

Step 9: Move Issues During Sprint

Simulate work:

Issue	Move To
Create user registration page	Done
Develop API for login	In Progress
Design home page UI	Done
Implement password reset	In Progress
Write unit test cases	To Do

Drag cards between columns.

Step 10: Complete Sprint (End of Sprint)

1. After moving tasks, click **Complete Sprint**.
2. Jira will close sprint and show report.

Step 11: Sprint Board at End (Screenshot 2)

Open completed sprint report.

Take Screenshot – Sprint Board at End

Result

Thus, a Scrum project was successfully created in Jira with backlog prioritization, sprint planning, sprint execution, and sprint completion.

10	Use the following requirements for a Library Management System: • Add a feature to search books by title and author. • Implement an online book reservation system. • Generate monthly reports on borrowed books for administrators. • Enable email notifications for overdue books. • Add support for QR code scanning for borrowing and returning books. • Create a user-friendly dashboard for librarians. • Allow users to review and rate books. • Integrate a chatbot for user assistance. • Develop a mobile app version of the system. • Provide multi-language support. Categorize each requirement using MOSCOW Method (Must-Have, Should-Have, Could-Have, or Won't-Have) based on the following criteria: • Impact on the users and stakeholders. • Feasibility considering time, budget, and resource constraints. Finally Submit the completed Google Sheet or Excel file with all requirements categorized and justified.	Google Sheet or Excel	CO2
----	---	-----------------------	-----

Objective

To categorize system requirements using the MoSCoW prioritization method based on user impact and feasibility, and document them in Google Sheets / Excel.

Step-by-Step Procedure (Google Sheets / Excel)

♦ Step 1: Open Spreadsheet

Open:

- Google Sheets → Blank Sheet
OR
- Excel → New Workbook

♦ Step 2: Create Column Headings

In Row 1, enter:

A1	B1	C1	D1	E1
----	----	----	----	----

Requirement ID	Requirement Description	MoSCoW Category	Justification	Priority Reason

◆ **Step 3: Enter Requirements**

Enter all given requirements:

ID	Requirement
R1	Search books by title and author
R2	Online book reservation
R3	Monthly borrowed books report
R4	Email notifications for overdue books
R5	QR code scanning
R6	Librarian dashboard
R7	Review and rate books
R8	Chatbot integration
R9	Mobile app version
R10	Multi-language support

◆ **Step 4: Apply MoSCoW Categorization**

Fill columns C, D, and E as follows:

Final Categorized Table

Req ID	Requirement	MoSCoW Category	Justification	Priority Reason
R1	Search books by title and author	Must-Have	Core library functionality	Essential for users

R2	Online book reservation	Must-Have	Improves accessibility	High user demand
R3	Monthly reports	Should-Have	Important for administration	Operational efficiency
R4	Email overdue notification	Must-Have	Prevents book loss	High impact
R5	QR code scanning	Could-Have	Enhances speed	Requires extra hardware
R6	Librarian dashboard	Must-Have	Required for management	Core system control
R7	Review and rating	Could-Have	Improves engagement	Optional feature
R8	Chatbot	Could-Have	Helps users	Resource intensive
R9	Mobile app version	Should-Have	Improves accessibility	Medium feasibility
R10	Multi-language support	Won't-Have	High cost and complexity	Low priority initially

♦ Step 5: Color Code (Optional but Recommended)

Apply colors:

- Must-Have → Green
- Should-Have → Blue
- Could-Have → Yellow
- Won't-Have → Red

This improves visual clarity.

♦ Step 6: Save File

Google Sheets:

File → Share → Copy link

Excel:

File → Save As → Library_MoSCoW.xlsx

Explanation of MoSCoW Method

Category	Meaning
Must-Have	Critical for system
Should-Have	Important but not urgent
Could-Have	Optional features
Won't-Have	Not in current version

Result

Thus, the Library Management System requirements were successfully categorized using the MoSCoW method and documented in Google Sheets / Excel.